实验 1. 基于分治法求解循环日程表问题

班级：数科 2102　　姓名：陈　　学号：202101

实验内容

本实验要求基于算法设计与分析的一般过程（即待求解问题的描述、算法设计、算法 描述、算法正确性证明、算法分析、算法实现与测试），a）利用分治法的循环日程表算法及 b）鼓励挑战 $n = 2^k$ 的情形， 以及分治法的其它经典应用实例如快速排序、归并排序、棋盘覆盖问题

实验目的

- 理解分治法的核心思想以及分治法求解过程。

实现语言

选择 C、C++、Java、Python 等之一。

循环日程表算法实验步骤

1、 理解问题，给出问题的描述。

对于$n = 2^k$位运动员的比赛，要求：

（1） 每个选手都要与其他$n-1$位选手进行有且仅有一次比赛

（2） 每天每位选手只能进行一次比赛

（3） 循环赛需进行$n-1$天

2、 算法设计，包括算法策略与数据结构的选择；

		D1	D2	D3	D4	D5	D6	D7
P1	1	2	3	4	5	6	7	8
P2	2	1	4	3	6	5	8	7
P3	3	4	1	2	7	8	5	6
P4	4	3	2	1	8	7	6	5
P5	5	6	7	8	1	2	3	4
P6	6	5	8	7	2	1	4	3
P7	7	8	5	6	3	4	1	2
P8	8	7	6	5	4	3	2	1

将所有的选手分为两半，n 位选手的比赛日程表就可由 n/2 设计的比赛日程表 决定；通过递归进行分割，直到只剩下 2 个选手时，比赛日程表的制定就变得很简单。上图展示了 8 位比赛选手参赛时的日程表。可以发现，通过将问题一步步分解 直至只剩 2 位选手时，可以很简单的安排出日程表，再将数据赋值给右下，然后加 上选手数，赋值给左下和右上，一步步合并就能得到最终问题的解

3、 描述算法。希望采用源代码以外的形式，如<u>伪代码</u>、流程图等；

```
Solve(A, n)
  if n = 1
    then A[ 1][1] = 1
    return


  Solve(A, n/2) //求解另一规模为 n/2 的问题
  Copy(n, A)


 Copy(n, A)
   m = n/2
   for i = 1 to m
     for j = 1 to m
       do A[i][j+m] = A[i+m][j] = A[i][j]+m
 //将左上的数据加上 m ，并复制到右上和左下
         A[i+m][j+m] =A[i][j] //把左上的数据复制到右下
```

4、 算法的正确性证明。需要这个环节，在理解的基础上对算法的正确性给予证明；

可知数组 A[n][n]分别沿两条对角线对称

初始化：A[1][1] = 1 ，对称

迭代：设第 k- 1 次回归后数组 A 满足对称条件，由语句

A[i][j+m] = A[i+m][j] = A[i][j]+m

A[i+m][j+m] = A[i][j]

可知第 k 次迭代后 A 依然满足对称条件

终止：迭代 n/2 次后终止，显然整个数组满足对称条件

5、 算法复杂性分析，包括时间复杂性和空间复杂性；

时间复杂度：

$$T(n) = \begin{cases} O(1), & n = 1 \\ O(n/2) + c(n/2)^2, & n > 1 \end{cases}$$

由递归树可得 $T(n) = O(n^2)$

空间复杂度：

由递归次数可知为：O(log n)

循环日程表算法实验结果

算法实现与测试，附上代码或以附件的形式提交，同时贴上算法运行结果截图；

```
#include <iostream>
using namespace std;

int n;
const int N = 110;
int graph[N][N];

void Copy(int n) {
	int m = n / 2;
	for (int i = 1; i <= m; i++) {
		for (int j = 1; j <= m; j++) {
			graph[i + m][j] = graph[i][j + m] = graph[i][j] + m;
			graph[i + m][j + m] = graph[i][j];
		}
	}
}

void Solve(int n) {
	if (n == 1) {
		graph[1][1] = 1;
		return;
	}
	Solve(n / 2);
	Copy(n);
}

void Print() {
	for (int i = 1; i < n; i++) {
		cout << '\t' << "Day" << i;
	}
	cout << endl;

	for (int i = 1; i <= n; i++) {
		for (int j = 1; j <= n; j++) {
			if (j == 1) {
				cout << graph[i][j];
			} else {
				cout << '\t' << graph[i][j];
			}
		}
		cout << endl;
```

```
        }
    }

    int main() {
        cin >> n;
        Solve(n);
        Print();

        return 0;
    }
```

```
8
        Day1    Day2    Day3    Day4    Day5    Day6    Day7
1       2       3       4       5       6       7       8
2       1       4       3       6       5       8       7
3       4       1       2       7       8       5       6
4       3       2       1       8       7       6       5
5       6       7       8       1       2       3       4
6       5       8       7       2       1       4       3
7       8       5       6       3       4       1       2
8       7       6       5       4       3       2       1
PS F:\Code>
```

快速排序算法实验步骤

1、 理解问题，给出问题的描述。

对 n 个元素进行排序

2、 算法设计，包括算法策略与数据结构的选择；

通过一趟扫描将待排序元素分割为三个部分，一是不大于基准元素的元素；二是基准元素；三是不小于基准元素的元素。再分别对第一、三个序列排序，直至整个数组排序完成

3、 描述算法。希望采用源代码以外的形式，如**伪代码**、流程图等；

```
QuickSort(A, low, high)
  if low < high
    then index = Partition(A, low, high) //获取基准元素下标
    QuickSort(A, low, index-1) //对左半数组排序
    QuickSort(A, index+1, high) //对右半数组排序

Partition(A, low, high)
  tmp = A[i]
  j = high, i = low
  while(true)
    while(A[j] > tmp) //从右边向左找比 tmp 小的元素
      j –
      if(i >= j) return j
    swap(A[j], A[i++])
    while(A[i] < tmp) //从左边向右找比 tmp 大的元素
      i ++
```

```
            if(i >= j) return j
        swap(A[j--], A[i])
        if(i >= j) return j
    return j
```

4、 算法的正确性证明。需要这个环节，在理解的基础上对算法的正确性给予证明；

初始：数组被分为三部分：小等于基准元素的元素，基准元素，大等于基准元素的元素，此时基准元素已排好序

迭代：对小等于基准元素的元素与大等于基准元素的元素分别进行排序，每次都会有基准元素的产生（位置正确的元素）

终止：当 low >= high 时，此时每个子问题中小等于基准元素的元素与大等于基准元素的元素最多只有一个，故已排好序，基准元素也已排好序，使得整个数组变得有序

5、 算法复杂性分析，包括时间复杂性和空间复杂性；

时间复杂度：

由递归树可得最坏情况下为 O(n^2)，最好与平均情况为 O(nlogn)

空间复杂度：

最坏 O(n)，最好与平均情况 O(logn)

快速排序算法实验结果

算法实现与测试，附上代码或以附件的形式提交，同时贴上算法运行结果截图**;**

```
#include <iostream>
using namespace std;

const int N = 1e3 + 10;
int n;
int a[7];

void Print(int* a, int n);
void QuickSort(int* a, int low, int high);
int Partition(int* a, int low, int high);

int main() {
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    QuickSort(a, 0, n - 1);
    Print(a, n);

    return 0;
}

void Print(int* a, int n) {
```

```
    for (int i = 0; i < n; i++) {
        if (i == 0)
            cout << a[i];
        else
            cout << " " << a[i];
    }
    cout << endl;
}

void QuickSort(int* a, int low, int high) {
    if (low < high) {
        int index = Partition(a, low, high);
        QuickSort(a, index + 1, high);
        QuickSort(a, low, index - 1);
    }
}

int Partition(int* a, int low, int high) {
    int i = low, j = high;
    int tmp = a[i];
    while (true) {
        while (a[j] > tmp){
            j--;
            if (i >= j)
                return j;
        }
        swap(a[j],a[i++]);

        while (a[i] < tmp){
            i++;
            if (i >= j)
                return j;
        }
        swap(a[j--],a[i]);
        if (i >= j) return j;
    }

    return j;
}
```

```
10
100 98 76 65 54 43 32 22 21 -2
-2 21 22 32 43 54 65 76 98 100
PS F:\Code>
```

实验总结

(1) 遇到哪些困难，如何解决的

暂无，若有可通过 DeBug 解决

(2) 有哪些收获

在面对规模大的问题时，若问题可被分解为规模更小的问题且最终解可表示为子问题解的合并，则可以考虑使用分治法解决问题

(3) 本章所学到算法设计方法，能否举一反三，进行拓展 分治法所能解决的问题一般具有以下几个特征：

1、该问题的规模缩小到一定的程度就可以容易地解决

2、该问题可以分解为若干个规模较小的相同问题

3、利用该问题分解出的子问题的解可以合并为该问题的解

4、该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题

这些特性的存在使分治法可以运用到二分查找，快速排序等问题求解上

对于子问题间有重复的，可判别其是否为可重复贡献问题，并使用 st 表解决

(5) 针对本章学习内容，还需要老师哪些方面的帮助。

暂无